# Nemotron Reasoning Challenge — Data-Centric Fine-Tuning

Fine-tuning **NVIDIA Nemotron-3-Nano-30B-A3B** (a 30-billion-parameter Mixture-of-Experts model) with LoRA to solve programmatically-generated reasoning puzzles, for the NVIDIA Nemotron Model Reasoning Challenge.

**Final result:** 0.84 on the private leaderboard from my own pipeline, up from a 0.51 base-model baseline — roughly the top 25% of ~4,000 teams.

---

## Table of contents



---

## The problem

The competition gives the model a few worked examples that demonstrate a hidden transformation rule, and asks it to apply that rule to a new input. There are nine puzzle families:

| Family | Rule | How solvable |
|---|---|---|

| | | |
|---|---|---|
| gravity | `d = 0.5*g*t^2` , g hidden | ~100% (deterministic) |
| unit_conversion | `out = rate*in` , rate hidden | ~100% (deterministic) |
| numeral | integer <-> Roman numeral | ~100% (deterministic) |
| cipher | letter substitution | ~100% (deterministic) |
| bit_manipulation | per-bit boolean gates | ~55% |
| equation_numeric_deduce | solve for unknowns | ~90% |
| equation_numeric_guess | underdetermined | ~15% |
| cryptarithm_deduce | letters = digits | ~8% |
| cryptarithm_guess | underdetermined | ~7% |

Scoring is **exact match at temperature 0** — the model must commit to a rule and execute it precisely. No partial credit, no room for hand-waving.

---

## My approach

**Data quality over training tricks.** If the chain-of-thought (CoT) a model trains on is internally consistent and verifiably correct, the model learns to *reason* rather than to *guess*. My work centers on generating training traces that are provably correct and never contradict themselves.

---

## What I built

1. **Deterministic reasoners** for the puzzle types that can be solved exactly from the prompt (gravity, unit conversion, Roman numerals). Each parses the prompt, recovers the hidden rule from the worked examples, verifies it against a second example, applies it to the target, and emits a step-by-step trace ending in the boxed answer.
2. **A verified synthetic-data generator** that creates *fresh* puzzles (own random parameters, not copied from the training set), pairs each with a CoT from the reasoners, and runs a hard verification gate that independently recomputes every answer and drops anything that doesn't match exactly.
3. **A LoRA fine-tuning pipeline** for the 30B MoE model using Unsloth, with the training

configuration that empirically held the score, correct adapter-key handling for the MoE backbone, and stratified batching across puzzle types.

---

## Every file in this repo

| File | What it is | Notes |
|---|---|---|
| `README.md` | This document | — |
| `generate_augmented.py` | The synthetic-data generator. Builds fresh gravity / unit / numeral puzzles, attaches verified CoT, runs the correctness gate, writes the CSV. | Run this to reproduce the dataset. Imports the three reasoner modules. |
| `reasoners/gravity.py` | Deterministic gravity solver ( `d = 0.5*g*t^2` ). | `parse()` + `reason()` returning `(cot, answer)` . |
| `reasoners/unit_conversion.py` | Deterministic linear-conversion solver. | Same interface. |
| `reasoners/numeral.py` | Roman-numeral solver, both directions, with round-trip verification. | Includes `to_roman()` / `from_roman()` helpers. |
| `train_nemotron.ipynb` | The Kaggle training notebook: environment setup for the Blackwell GPU, Unsloth model load, LoRA, SFT training, submission packaging. | Loads dgxchen base data **+** the augmented CSV. |
| `augmented_examples.csv` | 2,400 generated examples (800 each: gravity, unit_conversion, numeral). | Schema: `id, type, prompt, answer, generated_cot` . Every answer independently verified. |

> **Note on `reasoners/__init__.py`:** if you upload the reasoners through the GitHub web UI by typing `reasoners/gravity.py` as the filename, GitHub creates the folder automatically. To run `generate_augmented.py` locally as a package, add an empty `reasoners/__init__.py`.

---

## How to reproduce

**Generate the dataset locally:**

```
# from the repo root
python generate_augmented.py
# -> writes augmented_examples.csv (2,400 verified rows)
```

The generator uses a fixed random seed, so the dataset is reproducible, and the verification gate guarantees every written row recomputes to its stated answer.

**Train on Kaggle:**

1. Open `train_nemotron.ipynb` on Kaggle.
2. Attach inputs: the competition data, the Nemotron-3-Nano-30B model, the offline package wheels, the NVIDIA utility script, and `augmented_examples.csv` uploaded as a Kaggle dataset.
3. Set the GPU accelerator, Internet off, and Run All.
4. Submit the generated `submission.zip`.

---

## Score progression (my own pipeline)

| Stage | Private score |
|---|---|
| Base model, no fine-tuning | 0.51 |
| First SFT, short reasoning traces | 0.66 |
| Unsloth + corrected CoT format + MoE training | 0.84 |
| + my verified augmented data | 0.84 |

The augmented data did not raise the ceiling, because the three types it strengthens were

already near-perfect in the base training data. The remaining gap to a medal lives in the cryptarithm and equation-guess families, which even top teams solve only ~7-15% of the time — genuinely hard, not a tuning issue.

---

## The rounding-contradiction bug (the key insight)

This is the part I'm most proud of, and the thing worth understanding.

A common flaw in puzzle training data: the chain-of-thought computes one value in its reasoning (say `156.347`) but is then force-labeled with a rounded dataset answer (`156.35`). The trace literally says one number and then boxes a different one. Repeated across thousands of examples, this teaches the model that *its own derivation and its final answer don't need to agree* — which is poison for a task scored on exact match.

My generator avoids this by construction: **the answer is computed along the exact same rounded arithmetic path the CoT narrates.** The rate is rounded once, shown in the trace, and reused everywhere, so the boxed answer is always the number the reasoning actually arrives at. Then a verification gate re-derives every answer independently and drops any row that fails. The result is 2,400 examples with **zero contradictions**, confirmed by post-hoc check.

---

## On forked notebooks (honest note)

Public community notebooks reaching ~0.86 on the **public** leaderboard were widely shared during the competition. I ran several of them as **baselines** — to benchmark my own pipeline against the field and to understand the gap. I am documenting that openly rather than presenting other people's adapters as my own work.

It also produced a genuinely useful lesson about leaderboard dynamics. Final standings are scored on held-out **private** data, and there was a meaningful shakeup between public and private scores (my own selected run went 0.86 public -> 0.84 private). Optimizing hard against the visible public number — including by forking high-public-score adapters — did not reliably survive that shuffle. The genuine, generalizable pipeline is the part that holds up and the part worth keeping.

---

## What I would do next

- **Bit-manipulation reasoner** (bit-serial: solve each output bit independently from the 8-

bit truth tables). This is the single largest remaining score lever (~17% of the test set).

- **Equation-deduce reasoner** for the solvable subset (~8%).
- Treat the cryptarithm families as **constraint satisfaction** rather than pattern guessing.

---

## Tech stack

Python, PyTorch, Unsloth, PEFT / LoRA, Hugging Face Transformers and TRL, Kaggle (RTX PRO 6000 / Blackwell GPU).

---

*Built by Anjana Mohan. The reasoners and the verified data generator are my own implementation; the training scaffold follows community-shared best practices for this model, credited where used.*